

► Funciones

► Módulos

Bibliografía

- *Computational Physics*, Capítulo 2, Mark Newman.
- *Introducción a la programación con Python*. Capítulo 6, Andrés Marzal, Isabel Gracia y Pedro García.
- *Programming for computations- Python*, Capítulo 4 y sección: 5.4, Evein Linge and Petter Langtangen.
- *Computational mathematics: An introduction to Numerical Analysis and Scientific Computing with Python*. Sección: 1.5, Dimitrios Mitsotakis.

Funciones

Hasta el momento, utilizamos las funciones que traen definidas las librerías de Python. Por ejplo, `sqrt`, `cos`, `arcos`, por nombrar unas cuantas. Sin embargo, existen muchas situaciones en Física Computacional, en las que necesitamos definir nuestras propias funciones.

La estructura básica para escribir funciones en Python es:

```
1 def la_funcion(parametro1,parametro2=valor2,...):  
2     ''' Ayuda de la función '''  
3     declaraciones  
4     return valor_funcion  
5
```

El texto que se escriba en la '''Ayuda de la función''' sirve para indicarnos luego de que se trata la función y queda definido como el atributo del objeto: `la_funcion.__doc__` o `la_funcion?` en Ipython. El argumento opcional `valor2` es el valor que adquiere el parámetro si la función es llamada sin especificar el `parametro2`.

Por ejemplo, $f(x) = 8x^7 - x^3 + x^2 - 1$,

```
1 In [1]: def f(x):  
2     ...:     y = 8*x**7 - x**3 + x**2 -1  
3     ...:     return y  
4     ...:  
5  
6 In [2]: f(0)  
7 Out[2]: -1  
8
```

```
9 In [3]: f(-1)
10 Out[3]: -7
11
12 In [4]: def f(x):
13     ...:     return 8*x**7 - x**3 + x**2 -1
14     ...:
15
16 In [5]: f(-1)
17 Out[5]: -7
18
19 In [6]: f(0)
20 Out[6]: -1
```

Note que se puede omitir la definición de la variable y poniendo la operación después del `return`. En ocasiones, queremos entregar un valor por defecto a alguna de las variables. Por ejemplo,

```
1 In [1]: def f(x):
2     ...:     return 8*x**7 - x**3 + x**2 -1
3     ...:
4
5 In [2]: f(-1)
6 Out[2]: -7
7
8 In [3]: f(0)
9 Out[3]: -1
10 In [4]: def maximo(x,y=0):
11     ...:     if x >=y:
12     ...:         return x
13     ...:     else:
14     ...:         return y
15     ...:
16
17 In [5]: maximo(1)
18 Out[5]: 1
19
20 In [6]: maximo(2)
21 Out[6]: 2
22
23 In [7]: maximo(2),5
24 Out[7]: (2, 5)
25
26 In [8]: maximo(2,5)
27 Out[8]: 5
28
29 In [9]: maximo(7,5)
30 Out[9]: 7
31
32 In [10]: maximo(0,5)
33 Out[10]: 5
34
35 In [11]: maximo(0)
```

```

36 Out[11]: 0
37
38 In [12]: maximo(-1)
39 Out[12]: 0
40
41 In [13]: a=50; b=100;
42
43 In [14]: maximo(a,b)
44 Out[14]: 100

```

Como último ejemplo, queremos calcular el factorial de un número,

$$n! = 1 \times 2 \times 3 \times \dots (n-2) \times (n-1) \times n = \prod_{k=1}^n k.$$

Es decir, $3! = 1 \times 2 \times 3 = 6$.

factorial.py

```

1  """ Este es un ejemplo del uso de funciones """
2
3  def factorial(n):
4      ''' Calcula el factorial de un número entero'''
5      f = 1
6      for i in range(1,n+1):
7          f = i * f
8      return f
9
10 ### acá comenzamos el programa
11
12 a = int(input("ingrese el número entero del que quiere conocer el factorial
13 \n"))
14
15 if a>=0:
16     elifatorial = factorial(a)
17     print(f"El factorial del número que entraste es: {elifatorial} ")
18 else:
19     print("No es la función Gamma, el número debe ser entero mayor o igual a
20     cero.")

```

El ejemplo del cálculo del factorial nos permite analizar paso a paso los componentes esenciales de una función en Python y la dinámica de ejecución dentro del programa:

1. **Definición de la función (def):** La instrucción `def factorial(n):` le indica al intérprete que estamos creando un bloque de código reutilizable llamado `factorial`, el cual espera recibir un argumento llamado `n` (el parámetro de entrada).
2. **Documentación o Docstring:** La cadena dentro de triples comillas `'''Calcula el factorial de un número entero'''` constituye el *docstring*. Es una práctica fundamental en software científico, ya que permite consultar la ayuda del objeto mediante `factorial?` o `help(factorial)` en entornos interactivos como IPython o Jupyter.

3. **Variables locales y acumuladores:** Dentro de la función se define $f = 1$. Esta variable es de **ámbito local** (*local scope*), lo que significa que solo existe mientras la función se está ejecutando. El bucle `for i in range(1, n + 1):` actualiza el acumulador multiplicando $f = i * f$ (o equivalentemente $f *= i$) desde $i = 1$ hasta $i = n$.
4. **Valor de retorno (`return`):** La sentencia `return f` finaliza la ejecución de la función y devuelve el resultado computado al punto exacto del programa donde fue invocada. Sin un `return`, la función devolvería `None`.
5. **Flujo del programa principal y paso de parámetros:** En la sección principal del script:
 - Se captura la entrada del usuario mediante `input()` y se convierte explícitamente a entero con `int()`.
 - Se evalúa mediante una estructura condicional (`if a >= 0`) que el número sea no negativo.
 - En la línea `elfactorial = factorial(a)`, la variable externa `a` pasa su valor al parámetro local `n`. El resultado devuelto por `return` se almacena en la nueva variable `elfactorial`.
 - Finalmente, se despliega el resultado mediante una cadena formateada: `f"El factorial del número que entraste es: {elfactorial}"`.

Observación

Note que para usar la función debo hacerlo afuera de su definición, después del comentario `### acá comienza el programa`. Python permite el uso la estructura `if __name__ == "__main__":`. Lo veremos después cuando hablemos de módulos.

Ejemplo:

tiempo_vuelo_v3.py

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 def altura(y0, v0, t):
6     """Esta función entrega la altura de una bola en función del tiempo'
7     Parametros: altura inicial y0, velocidad inicial v0 y tiempo"""
8     g = 9.8 # aceleración gravitacional de la tierra en m/s^2
9     return y0 + v0 * t - 1/2. * g * t**2
10
11
12 #--- programa principal -----
13 print('-----')
14 y0 = float(input('Entre la altura de lanzamiento en metros\t'))
15 v0 = float(input('Entre la rapidez con la que lo lanzas en m/s\t'))
16 N = int(input("Entre el número de puntos para el paso del tiempo\t"))
17 print('-----')
18 g = 9.8 # aceleración gravitacional de la tierra en m/s^2
19

```

```

20 #----- solucion exacta -----
21 T = (v0 + np.sqrt( v0**2 + 2*g*y0) )/g
22 #-----
23
24 t_i = 0.0 # tiempo inicial
25 t_f = 10. #tiempo final
26 t = np.linspace(t_i, t_f, N) # arreglo con los tiempos
27
28 # llamamos a la función altura
29 y = altura(y0,v0,t)
30
31 # con el while buscamos que esa altura siempre sea positiva.
32 i = 0
33 while y[i] >=0:
34     i = i +1
35
36 t_num=t[i]
37
38 # calculo de errores
39
40 err_abs = abs(t_num - T)
41 err_rel = (err_abs / T) * 100
42
43 print(f"El tiempo que queda la bola en el aire: {t_num:.4f} segundos")
44 print(f"El tiempo exacto en el que cae la bola es : {T:.4f} segundos")
45 print("\n--- ANÁLISIS DE ERROR Y PRECISIÓN ---")
46 print(f"Error absoluto (|t_num - T|) : {err_abs:.6f} s")
47 print(f"Error relativo (%) : {err_rel:.4f} %")
48 print(f"La altura para ese tiempo es: {y[i]:.4e} metros")

```

Note que la aceleración gravitacional se definió en el interior de la función, esto la convierte en una variable local. Eso significa que, para calcular la solución exacta $T = (v_0 + \sqrt{v_0^2 + 2gy_0})/g$ debo volver a definir g afuera de la función. Mucho cuidado con el orden en que se entran los argumentos de la función, `altura(v0,y0,t)` no genera error pero devuelve un resultado incorrecto físicamente.

Para ilustrar el concepto de las variables locales y globales, considere el siguiente ejemplo

```

1 In [16]: def variables(x,y):
2     ...:     temp = x
3     ...:     x = y
4     ...:     y= temp
5     ...:     print('Estas son variables locales: x =',x, 'y=',y)
6     ...:     return x,y
7
8 In [17]: x = 2; y= 3
9
10 In [18]: variables(x,y);
11 Estas son variables locales: x = 3 y= 2
12
13 In [19]: print("Estas son las variables globales: x=",x,'y=',y)
14 Estas son las variables globales: x= 2 y= 3

```

La función Lambda

En ocasiones simplemente queremos definir una función corta, como por ejemplo $f(x) = x^3 - x$ o $g(x) = x^2$.

```
1 In [1]: g = lambda x: x**2
2
3 In [2]: g(-1)
4 Out[2]: 1
5
6 In [3]: g(1)
7 Out[3]: 1
8
9 In [4]: g(45)
10 Out[4]: 2025
11
12 In [5]: def g(x): # equivalente al uso de lambda
13     ...:     return x**2
14     ...:
```

Módulos en Python

Un módulo es un archivo que contiene un conjunto de funciones y su estructura está dada por la siguiente sintaxis:

```
1 def la_funcion1(parametro1,parametro2=valor2,...):
2     ''' Ayuda de la función '''
3     declaraciones
4     return valor_funcion
5
6 def la_funcion2(parametro1,parametro2=valor2,...):
7     ''' Ayuda de la función '''
8     declaraciones
9     return valor_funcion
10
11 def la_funcion3(parametro1,parametro2=valor2,...):
12     ''' Ayuda de la función '''
13     declaraciones
14     return valor_funcion
15 .....
16
17 if __name__ == "__main__":
18     Código de prueba o test del módulo
19
```

En Python viene una variable definida internamente `__name__` Esta variable toma el valor del programa cuando se carga como módulo o el valor cuando se corre como programa.

Veamos por ejemplo como se vería la función para el cálculo del factorial,

factorial_v2.py

```
1 """ Este es un ejemplo del uso de funciones como módulo para otros programas
2 Devuelve el factorial de n entero """
3
4 def factorial(n):
5     ''' Calcula el factorial de un número entero n! = n*(n-1)*(n-2)...'''
6     f = 1.
7     for i in range(1,n+1):
8         f = i * f
9     return f
10
11
12 #----- programa principal -----
13 if __name__ == "__main__":
14     a = int(input("ingrese el número entero del que quiere conocer el
15     factorial\n"))
16
17     if a>=0:
18         elfactorial = factorial(a)
19         print(f"El factorial del número que entraste es: {elfactorial} ")
20     else:
21         print("No es la función Gamma, el número debe ser entero mayor o
22         igual a cero.")
```

El Punto de Entrada: `if __name__ == "__main__":`

En nuestros programas es fundamental que los de código sean modulares, es decir, que puedan funcionar tanto como programas independientes que se ejecutan directamente, como librerías de funciones importables desde otros archivos.

Para lograr este comportamiento, Python utiliza una variable especial interna llamada `__name__`.

1. ¿Qué es `__name__` y cómo funciona?

Cada vez que el intérprete de Python ejecuta un archivo de código (.py), asigna automáticamente un valor a la variable oculta `__name__` según el contexto:

- **Si el archivo se ejecuta directamente** (por ejemplo, corriendo `python factorial.py` en la terminal): Python le asigna a `__name__` la cadena exacta `"__main__"`.
- **Si el archivo es importado como módulo** dentro de otro programa (por ejemplo, haciendo `import factorial` desde otro script): Python le asigna a `__name__` el nombre real del archivo (en este caso, `"factorial"`).

2. El propósito del condicional

Al escribir la estructura:

```

1 if __name__ == "__main__":
2     # Código del programa principal
3     a = int(input("Ingrese un número: "))
4     print(factorial(a))

```

Le estamos diciendo a Python:

*«Ejecuta este bloque de código **únicamente** si el archivo está siendo ejecutado de forma directa. Si el archivo está siendo importado por otro script, ignora este bloque y limita tu trabajo a ofrecer las funciones definidas arriba.»*

Veamos un ejemplo utilizando la función factorial para calcular los coeficientes binomiales

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad \text{para } 0 \leq k \leq n,$$

y el triángulo de Pascal.

triangulo_pascal.py

```

1 """ Lee la función factorial, calcula la combinatoria y muestra el triángulo
2 de Pascal """
3
4 import factorial_v2 as fa # importamos nuestra función
5
6 def binomial(n, k):
7     """
8     Calcula el coeficiente binomial C(n, k) = n! / (k! * (n - k)!)
9     usando la función factorial previamente definida.
10    Retorna un número entero.
11    """
12    if k < 0 or k > n:
13        return 0
14    if k == 0 or k == n:
15        return 1
16
17    # Usamos la función factorial que llamamos
18    num = fa.factorial(n)
19    den = fa.factorial(k) * fa.factorial(n - k)
20    return int(num/den)
21
22 def imprimir_pascal(filas):
23     """Imprime las primeras filas del Triángulo de Pascal."""
24     for n in range(filas):
25         linea = []
26         for k in range(n + 1):
27             # Llamamos a la función de coeficientes binomial(n, k)
28             coeficiente = binomial(n, k)
29             linea.append(str(coeficiente))
30
31     # Unir los números con espacios sin centrar

```

```

32     #print(" ".join(linea))
33     # Centra el texto en un ancho fijo de 40 caracteres
34     print(" ".join(linea).center(40))
35
36 if __name__ == "__main__":
37
38     # Pedir las filas que desee del triángulo de Pasca
39     n = int(input("¿Cuántas filas desea para el triángulo?\n"))
40     imprimir_pascal(n)

```

1. Importación de Módulos Propios y Alias (`import ... as`):

- La instrucción `import factorial_v2 as fa` busca el archivo `factorial_v2.py` en el mismo directorio de trabajo e importa todas sus funciones.
- El uso del alias `as` nos permite invocar la función con una sintaxis más corta y limpia: `fa.factorial(n)` en lugar de `factorial_v2.factorial(n)`.

2. Definición de `binomial(n, k)`:

- Se incluyen optimizaciones con casos base: si $k < 0$ o $k > n$, el coeficiente es 0; si $k = 0$ o $k = n$, devuelve directamente 1 sin necesidad de calcular factoriales ($\binom{n}{0} = \binom{n}{n} = 1$).
- Nota de precisión:** La línea `return int(num / den)` o idealmente la división entera `return num // den` garantiza que el resultado sea de tipo entero (`int`), evitando decimales innecesarios en la impresión (e.g., 6 en lugar de 6,0).

3. Construcción de la Cadena :

- `linea.append(str(coeficiente))`: Convierte cada número que se tiene en la variable `coeficiente` en cadena para almacenarlo en la lista temporal `linea`.
- `" ".join(linea)`: Concatena los elementos de la lista intercalando un espacio entre ellos (ejemplo: `['1', '3', '3', '1']` $\rightarrow$ `"1 3 3 1"`).
- `.center(40)`: Es un método de cadenas de Python que añade espacios en blanco de forma equitativa a la izquierda y derecha para centrar la fila dentro de un bloque fijo de 40 caracteres de ancho, generando la geometría piramidal clásica del triángulo.

Ejemplo: secuencia de Fibonacci

Veamos un ejemplo de un módulo que me genera los números de Fibonacci.

La siguiente discusión fue tomada del libro: [Introduction to real analysis, Robert G. Bartle et al](#)

La secuencia de Fibonacci es una sucesión infinita de números naturales que comienza con 1, y cada número subsiguiente es la suma de los dos anteriores. La secuencia se define de la siguiente manera, $F := (f_n)_{n \in \mathbb{N}}$

$$f_1 := 1, \quad f_2 := 1, \quad f_{n+1} := f_{n-1} + f_n, \quad (n \geq 2). \quad (1)$$

Note que los primeros números de Fibonacci son: 1, 1, 2, 3, 5, 8, 13... Dentro de muchas propiedades de la sucesión de Fibonacci, se encuentra que el cociente de dos números consecutivos tiende a la razón áurea

$\phi = \frac{1+\sqrt{5}}{2} \approx 1,6180339887$ (para más detalles ver el siguiente video: [La sucesión de fibonacci y la razón áurea](#)) Acá te dejo otros dos videos interesantes sobre la sucesión de Fibonacci: [Lo que nadie sabe de la sucesión de Fibonacci](#) y [Las plantas y fibonacci](#). También te dejo un [artículo de divulgación](#).

Vamos entonces a probar que $\lim_{n \rightarrow \infty} \frac{f_{n+1}}{f_n} = \phi$. Para ello partimos de la ecuación (1) y dividimos ambos lados por f_n para obtener

$$\frac{f_{n+1}}{f_n} = \frac{f_{n-1}}{f_n} + 1. \quad (2)$$

Si definimos $x_n = \frac{f_{n+1}}{f_n}$, entonces la ecuación anterior se puede reescribir como

$$x_n = \frac{1}{x_{n-1}} + 1. \quad (3)$$

Asumiendo que la sucesión $(x_n)_{n \in \mathbb{N}}$ converge a L , se tiene que

$$\lim_{n \rightarrow \infty} x_n = \lim_{n \rightarrow \infty} \left[\frac{1}{x_n} + 1 \right] \Rightarrow L = \frac{1}{L} + 1 \Rightarrow L^2 - L - 1 = 0. \quad (4)$$

La ecuación anterior se resuelve de la manera usual y se encuentran dos raíces. La raíz positiva coincide con la razón áurea $L = \frac{1 + \sqrt{5}}{2} \equiv \phi$.

A continuación, mostraremos numéricamente el resultado anterior.

fibonacci.py

```
1 """
2 fibonacci.py
3 Es un módulo que genera los números de Fibonacci y se verifica la famosa
4   razón áurea.
5 """
6
7 import math
8
9 def Fibonacci(N):
10     """
11     Genera y devuelve una lista con los primeros N números de Fibonacci.
12
13     Parametros de entrada:
14     N (int): Número de Fibonacci que desea generar.
15
16     Devuelve:
17     list: Los primeros N números de Fibonacci comenzando desde f_1 = 1 y f_2
18           = 1.
19     """
20     if N <= 0:
21         print("Error: N debe ser un entero positivo.")
22         return []
23     elif N == 1:
24         return [1]
```

```
22
23     # Inicia la secuencia (lista) con los dos primeros números
24     fib_sequence = [1, 1]
25
26     # Agrego los números de Fibonacci hasta que la longitud de la lista sea
27     # N, con el .append
28     # Tomamos el último de la lista fib_sequence[-1]
29     # Tomamos el penúltimo de la lista fib_sequence[-2]
30     # Agregamos cada elemento a la lista fib_sequence.append(fib_sequence
31     [-1] + fib_sequence[-2])
32     # Devuelve la lista desde e primero hasta el N-1 fib_sequence[:N]
33     for i in range(2, N):
34         fib_sequence.append(fib_sequence[-1] + fib_sequence[-2])
35
36     return fib_sequence[:N]
37
38 def convergencia(F):
39     """
40     Acá chequeamos la convergencia de la sucesión de Fibonacci y comparamos
41     con la razón áurea.
42
43     Paramétro de entrada: La lista F con los números de Fibonacci.
44
45     Retorna:
46     None: Imprime una tabla con la comparación entre la convergencia de la
47     sucesión de fibonacci y la razón áurea.
48     """
49     # La razón áurea: (1 + sqrt(5)) / 2
50     aurea = (1 + math.sqrt(5)) / 2
51
52     # Se necesitan al menos dos números de Fibonacci para calcular la razón
53     valid_fibs = [x for x in F if x > 0]
54
55     if len(valid_fibs) < 2:
56         print("Error: La lista de entrada debe contener al menos dos números
57         de Fibonacci.")
58         return
59
60     print(f"{'n':<5} | {'f_n':<10} | {'f_{n+1}':<10} | {'Razón (F_{n+1} / F_n)
61     ':<22} | {'Diferencia con la razón áurea':<30}")
62     print("-" * 85)
63
64     ratios = []
65     for i in range(len(valid_fibs) - 1):
66         fn = valid_fibs[i]
67         fn1 = valid_fibs[i+1]
68         ratio = fn1 / fn
69         ratios.append(ratio)
70         diff = abs(ratio - aurea)
```

```

68     print(f"{i+1:<5} | {fn:<10} | {fn1:<10} | {ratio:<22.10f} | {diff
        :<30.10e}")
69
70     # Chequea si la última diferencia es cercana a 0
71     final_diff = abs(ratios[-1] - aurea)
72     print("\n" + "=" * 85)
73     print(f"Razón áurea (Valor real): {aurea:.10f}")
74     print(f"Última razón calculada:      {ratios[-1]:.10f}")
75     print(f"Discrepancia absoluta:      {final_diff:.10e}")
76
77     if final_diff < 1e-4:
78         print("\nVerdicto: Johannes Kepler estaba correcto! .")
79     else:
80         print("\nVerdicto: La secuencia es demasiado corta para mostrar una
        convergencia.")
81
82
83 # --- Programa principal ---
84 if __name__ == "__main__":
85
86     print("--- Test Block del programa fibonacci.py ---\n")
87     N = int(input("Ingrese el número de términos para la secuencia "))
88
89     print(f"Generando los primeros {N} números de Fibonacci:")
90
91     fib_list = Fibonacci(N)
92     print(fib_list)
93     print("\n" + "=" * 85 + "\n")
94
95     print("Verificando la convergencia :")
96     convergencia(fib_list)

```

Problemas:

1. Defina una función que devuelva el valor de la raíz cúbica de un número dado.
2. Define una función que convierta grados centígrados a Kelvin y a Farenheit.
3. Defina una función que convierta grados en radianes.
4. Defina una función que daods dos parámetros, b y x , devuelva al logartimo en base b de x , $\log_b(x)$.
5. **Números perfectos:** Se dice que un número es perfecto si es igual a la suma de todos sus divisores, excpeto él mismo. Por ejemplo: los divisores de 28 son 1, 2, 4, 7, 14 y 28. Ahora, nota que $28 = 1 + 2 + 4 + 7 + 14$. Entonce s28 es un número perfecto. Diseña una función que nos diga si un número dado es perfecto o no.

6. Construcción del módulo *cinematica.py*

Para resolver el siguiente problema, *lea la sección*

a) **Parte 1: Diseño del Módulo (*cinematica.py*)** Cree un archivo llamado *cinematica.py* y defina las siguientes funciones:

- *posicion_1d*(*x0*, *v0*, *a*, *t*): Devuelve la posición $x(t)$. Debe aceptar tanto números flotantes como arreglos de NumPy para la variable t .
- *velocidad_1d*(*v0*, *a*, *t*): Devuelve la velocidad $v(t)$.
- *tiempo_vuelo_parabolico*(*v0*, *angulo_deg*, *y0*=0.0, *g*=9.8): Recibe la velocidad inicial, el ángulo en grados (convertir internamente a radianes con `np.radians`) y la altura inicial. Retorna el tiempo total que el proyectil permanece en el aire.
- *alcance_maximo*(*v0*, *angulo_deg*, *y0*=0.0, *g*=9.8): Hace uso interno de la función *tiempo_vuelo_parabolico* para retornar el alcance horizontal total R .

Requisito: Incluya el bloque `if __name__ == "__main__":` al final de *cinematica.py* con las pruebas necesarias para validar que sus funciones entregan valores físicos coherentes (por ejemplo, verificar que con $v_0 = 10$ m/s, $\theta = 45^\circ$, $y_0 = 0$, el alcance es aproximadamente 10,2 m).

b) **Parte 2: Aplicación en script principal** En un archivo separado ejecute las siguientes tareas importando su módulo recién creado:

- 1) Un cañón situado en un acantilado a $y_0 = 35$ m dispara proyectiles a $v_0 = 40$ m/s. Varíe el ángulo de lanzamiento θ desde 10° hasta 80° en pasos de 1° .
- 2) Usando las funciones de su módulo, encuentre numéricamente el ángulo óptimo que maximiza el alcance horizontal R y el tiempo de vuelo correspondiente.
- 3) Imprima los resultados en pantalla con formato profesional utilizando *f-strings*.
- 4) Haga gráficos de posición y velocidad contra el tiempo para diferentes ángulos. Específicamente:
 - Grafique $x(t)$ e $y(t)$ en función del tiempo para los ángulos $\theta \in \{15^\circ, 30^\circ, 45^\circ, 60^\circ, 75^\circ\}$.
 - Grafique la velocidad horizontal $v_x(t)$ y la velocidad vertical $v_y(t)$ en función del tiempo para los mismos ángulos, identificando el punto donde $v_y(t) = 0$ (altura máxima).

7. **Módulo de Geometría Vectorial (*vectores.py*):** Diseñe un nuevo módulo llamado *vectores.py* que contenga las siguientes funciones para vectores en 2D/3D representados como listas o arreglos de NumPy:

- *norma*(*v*): Devuelve la magnitud $|\vec{v}|$.
- *producto_punto*(*u*, *v*): Devuelve el escalar $\vec{u} \cdot \vec{v}$.
- *angulo_entre*(*u*, *v*): Devuelve el ángulo en grados entre ambos vectores.

Incluya casos de prueba en el condicional `if __name__ == "__main__":` para verificar que los vectores ortogonales devuelvan un ángulo de 90° .

8. **Caída Libre con fricción lineal:** La velocidad de una partícula en caída libre sujeta a resistencia del aire proporcional a la velocidad está dada por la solución analítica:

$$v(t) = \frac{mg}{c} (1 - e^{-\frac{c}{m}t})$$

donde m es la masa, $g = 9,8 \text{ m/s}^2$ y c es el coeficiente de arrastre.

- Cree una función `velocidad_terminal(m, c)` que retorne la velocidad límite $v_t = \frac{mg}{c}$.
- Cree la función `velocidad_friccion(t, m, c)` que evalúe la ecuación completa.
- Escriba el bloque `if __name__ == "__main__":` para pedir al usuario la masa m y el coeficiente c , e imprimir cuánto tiempo tarda la partícula en alcanzar el 99 % de su velocidad terminal.

Fundamentos Teóricos: Cinemática en 1D y 2D

Antes de implementar nuestro propio módulo de cinemática, es imperativo repasar las ecuaciones analíticas que gobiernan el movimiento de una partícula bajo aceleración constante a :

1. Movimiento en Una Dimensión (1D) con Aceleración Constante: Para una partícula que se mueve a lo largo de un eje con posición inicial x_0 , velocidad inicial v_0 y aceleración constante a :

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2 \quad (5)$$

$$v(t) = v_0 + a t \quad (6)$$

2. Movimiento de Proyectiles (2D) en un Campo Gravitacional: Cuando un proyectil se lanza desde una posición (x_0, y_0) con una rapidez inicial v_0 y un ángulo θ respecto a la horizontal, el movimiento se desacopla en dos ejes independientes (asumiendo aceleración gravitacional $g = 9,8 \text{ m/s}^2$ apuntando hacia abajo):

- **Eje Horizontal (x):** Movimiento a velocidad constante ($a_x = 0$).

$$v_x = v_0 \cos \theta, \quad x(t) = x_0 + (v_0 \cos \theta) t \quad (7)$$

- **Eje Vertical (y):** Caída libre con aceleración constante ($a_y = -g$).

$$v_y(t) = v_0 \sin \theta - g t, \quad y(t) = y_0 + (v_0 \sin \theta) t - \frac{1}{2} g t^2 \quad (8)$$

3. Magnitudes Características del Tiro Parabólico: Resolviendo $y(t) = 0$ para el tiempo de vuelo T_{vuelo} y evaluándolo en $x(t)$, obtenemos el alcance horizontal máximo x_{max} :

$$T_{\text{vuelo}} = \frac{v_0 \sin \theta + \sqrt{(v_0 \sin \theta)^2 + 2 g y_0}}{g} \quad (9)$$

$$x_{\text{max}} = x(T_{\text{vuelo}}) = x_0 + (v_0 \cos \theta) T_{\text{vuelo}} \quad (10)$$

4. Altura máxima del tiro parabólico: Resolviendo $v_y(t) = 0$ para el tiempo y evaluándolo en $y(t)$, obtenemos la altura máxima y_{max} .